راهنمای تشریح خط‌به‌خط

فایل notebooks/experiment.ipynb

پروژه SimSiam روی CIFAR-10

گزارش فارسی راست‌به‌چپ برای ارائه دانشگاهی

راهنمای استفاده از این گزارش

این سند دقیقاً بر اساس سلول‌های فایل `experiment.ipynb` نوشته شده است. notebook شامل سلول‌های Markdown برای توضیح علمی و سلول‌های Code برای اجرای واقعی است. متن‌های فارسی با HTML و ویژگی `dir="rtl"` راست‌به‌چپ می‌شوند، اما قطعه‌کدها عمداً چپ‌به‌راست باقی می‌مانند تا خوانایی Python حفظ شود.

> **نکته ارائه:** سلول‌ها را از بالا به پایین اجرا کنید. سلول آموزش طولانی با متغیر `RUN_TRAINING` کنترل می‌شود و در حالت پیش‌فرض خاموش است.

ساختار کلی notebook

بخش	هدف
سلول‌های ۱ و ۲	عنوان، هدف و توضیح علمی پروژه
سلول ۳	تنظیم مسیر پروژه، importها و انتخاب CPU/GPU
سلول‌های ۴ و ۵	تست معماری با داده ساختگی
سلول‌های ۶ و ۷	بارگذاری و اعتبارسنجی CIFAR-10
سلول‌های ۸ و ۹	pretraining خودنظارتی
سلول‌های ۱۰ و ۱۱	linear evaluation
سلول‌های ۱۲ و ۱۳	ارزیابی و ذخیره گزارش
سلول ۱۴	رسم loss
سلول‌های ۱۵ و ۱۶	تست خطای checkpoint
سلول‌های ۱۷ و ۱۸	اجرای Gradio demo

سلول ۱: عنوان و جهت متن

نوع: Markdown | نقش: معرفی notebook

```
<div dir="rtl" style="text-align:right">
# ارائه کامل پروژه SimSiam روی CIFAR-10
این notebook مسیر کامل پروژه را نشان می‌دهد.
</div>
```

تشریح خط‌به‌خط

- `<div>` یک بلوک HTML در Markdown می‌سازد.
- `"dir="rtl` جهت نمایش متن را راست‌به‌چپ می‌کند.
- `"style="text-align:right` تراز متن را سمت راست قرار می‌دهد.
- عنوان، موضوع ارائه و دیتاست استفاده‌شده را مشخص می‌کند.

سلول ۲: هدف پروژه

نوع: Markdown | نقش: بیان مسئله

این سلول تفاوت دو مرحله را توضیح می‌دهد: در pretraining مدل بدون label نمایش تصویری یاد می‌گیرد؛ در linear evaluation، encoder ثابت می‌شود و classifier خطی با label آموزش می‌بیند. این تفکیک برای نشان دادن کیفیت representation SimSiam مهم است.

سلول ۳: آماده‌سازی محیط

نوع: Code | نقش: import، مسیر پروژه و انتخاب device

```
import sys
from pathlib import Path
import torch
import matplotlib.pyplot as plt

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from utils.seed import seed_everything
seed_everything(42)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Project root:', PROJECT_ROOT)
print('PyTorch:', torch.__version__)
print('Device:', DEVICE)
```

تشریح خط‌به‌خط

خط	توضیح
`import sys`	برای دسترسی به مسیرهای import پایتون استفاده می‌شود.
`Path`	کار با مسیر فایل‌ها را مستقل از سیستم‌عامل می‌کند.
`torch`	کتابخانه اصلی tensor، شبکه عصبی و آموزش است.
`matplotlib`	برای رسم نمودار loss در انتهای notebook است.
`PROJECT_ROOT`	اگر notebook از داخل پوشه notebooks اجرا شود، یک سطح به ریشه پروژه برمی‌گردد؛ در غیر این صورت مسیر جاری را ریشه فرض می‌کند.
`sys.path.insert`	ریشه را به مسیر import اضافه می‌کند تا importهایی مانند `from ...models` کار کنند.
`seed_everything(42)`	seed تصادفی Python، NumPy و PyTorch را ثابت می‌کند تا نتیجه قابل تکرارتر باشد.
`DEVICE`	اگر CUDA موجود باشد GPU وگرنه CPU انتخاب می‌شود. پروژه روی CPU نیز اجرا می‌شود.

`print`	مسیر، نسخه PyTorch و سخت‌افزار را برای کنترل اولیه چاپ می‌کند.

سلول ۴: توضیح تست معماری

نوع: Markdown | نقش: توضیح ورودی و loss

در این بخش توضیح داده می‌شود که دو view از یک تصویر به projector، encoder و predictor می‌روند و stop-gradient در loss اعمال می‌شود. این سلول آموزشی است و اجرا نمی‌شود.

سلول ۵: تست dummy batch

نوع: Code | نقش: اطمینان از سالم بودن معماری قبل از آموزش

```
from models.simsiam import SimSiam
simsiam = SimSiam().to(DEVICE)
x1 = torch.rand(4, 3, 32, 32, device=DEVICE)
x2 = torch.rand(4, 3, 32, 32, device=DEVICE)
loss = simsiam(x1, x2)
assert loss.ndim == 0
print('Input:', tuple(x1.shape))
print('Loss:', round(loss.item(), 4))
print('Architecture test: PASS')
```

تشریح

- `SimSiam()` مدل کامل را می‌سازد.
- `.to(DEVICE)` مدل را روی CPU یا GPU قرار می‌دهد.
- `torch.rand(4,3,32,32)` چهار تصویر ساختگی رنگی ۳۲×۳۲ می‌سازد.
- `simsiam(x1,x2)` forward و loss دو view را اجرا می‌کند.
- `assert loss.ndim == 0` بررسی می‌کند loss یک مقدار اسکالر است و برای backward مناسب است.

سلول ۶: آماده‌سازی CIFAR-10

نوع: Markdown | نقش: توضیح محل داده و LIMIT

این متن می‌گوید پوشه باید در `data/cifar-10-batches-py` باشد. مقدار LIMIT برای اجرای سریع notebook است؛ مقدار ۵۰۰۰۰ برای کل تصاویر آموزشی استفاده می‌شود.

سلول ۷: خواندن و اعتبارسنجی داده

نوع: Code | نقش: ساخت dataset و بررسی شکل نمونه

```
from datasets.cifar10 import CIFAR10, CLASSES
LIMIT = 256
train_set = CIFAR10('data', train=True, download=False, two_views=True, limit=LIMIT)
test_set = CIFAR10('data', train=False, download=False, limit=LIMIT)
view1, view2, label = train_set[0]
assert tuple(view1.shape) == (3, 32, 32)
print('Train samples:', len(train_set))
print('Test samples:', len(test_set))
print('Classes:', ', '.join(CLASSES))
print('Sample:', tuple(view1.shape), 'label=', label)
```

تشریح خطبه‌خط

دستور	کارکرد
`...from datasets.cifar10`	کلاس dataset و نام کلاس‌های CIFAR-10 را import می‌کند.
`LIMIT = 256`	برای تست notebook فقط ۲۵۶ نمونه می‌خواند.
`train=True`	پنج فایل data_batch آموزشی را انتخاب می‌کند.
`two_views=True`	دو augmentation از هر تصویر برای SimSiam برمی‌گرداند.
`download=False`	داده را از اینترنت نمی‌گیرد و انتظار دارد قبلاً در data قرار گرفته باشد.
`train=False`	فایل test_batch را برای ارزیابی می‌خواند.
`train_set[0]`	اولین نمونه، دو view و label آن را می‌گیرد.
`assert`	شکل تصویر را کنترل می‌کند تا خطای ابعاد زود مشخص شود.

سلول‌های ۸ و ۹: pretraining

> **هدف: یادگیری representation بدون label**

سلول Markdown مفهوم pretraining را شرح می‌دهد. سلول Code تابع `train` را import می‌کند و متغیر کنترل‌کننده آموزش را تعریف می‌کند:

```
from training.pretrain import train as pretrain
RUN_TRAINING = False
if RUN_TRAINING:
    pretrained_model, pretrain_history = pretrain(
        epochs=5, batch_size=64, limit=50000, device=DEVICE,
        checkpoint='checkpoints/notebook_simsiam.pt'
    )
else:
    pretrained_model, pretrain_history = None, []
    print('Pretraining skipped. Existing checkpoint can be used.')
```

- `train as pretrain` نام طولانی تابع را برای خوانایی کوتاه می‌کند.
- `RUN_TRAINING=False` از اجرای ناخواسته و طولانی جلوگیری می‌کند.
- با True، پنج epoch و batch size برابر ۶۴ اجرا می‌شود.
- `limit=50000` یعنی تمام تصاویر آموزش CIFAR-10.
- `checkpoint` مسیر ذخیره وزن SimSiam است.
- در حالت False، checkpoint موجود برای مراحل بعد استفاده می‌شود.

سلول‌های ۱۰ و ۱۱: linear evaluation

هدف: سنجش کیفیت representation با classifier خطی

```
from training.linear_eval import train as linear_train
if RUN_TRAINING:
    classifier = linear_train(
        epochs=5, batch_size=64, limit=50000, device=DEVICE,
        encoder_checkpoint='checkpoints/simsiam.pt',
        checkpoint='checkpoints/linear.pt'
    )
else:
    classifier = None
    print('Linear training skipped. Existing checkpoint can be evaluated.')
```

`linear_train` ابتدا encoder SimSiam را از checkpoint بارگذاری و freeze می‌کند. سپس خروجی ۵۱۲بعدی encoder را به یک لایه خطی با ۱۰ خروجی می‌دهد. فقط classifier با labelها آموزش می‌بیند. این آزمایش نشان می‌دهد representation بدون label تا چه حد برای طبقه‌بندی مفید است.

سلول‌های ۱۲ و ۱۳: ارزیابی واقعی

هدف: محاسبه accuracy و ذخیره JSON

```
from evaluation.evaluate import evaluate_linear, evaluate_supervised, save_report
linear_result = evaluate_linear(limit=10000, device=DEVICE)
supervised_result = evaluate_supervised(limit=10000, device=DEVICE)
report = {'linear': linear_result, 'supervised': supervised_result}
save_report(report, 'results/notebook_evaluation.json')
print(f"Linear accuracy: {linear_result['accuracy']:.2%}")
print(f"Supervised accuracy: {supervised_result['accuracy']:.2%}")
```

خط	توضیح
import	دو ارزیاب و تابع ذخیره گزارش را می‌آورد.
`evaluate_linear`	encoder و classifier خطی را روی test set اجرا می‌کند.
`evaluate_supervised`	مدل supervised کامل را ارزیابی می‌کند.
`report`	نتایج دو روش را در یک dictionary می‌گذارد.

`save_report`	نتیجه را به JSON قابل استفاده در گزارش تبدیل می‌کند.
`2%.:`	accuracy اعشاری را به درصد با دو رقم اعشار نمایش می‌دهد.

سلول ۱۴: رسم loss

> **هدف: نمایش روند یادگیری pretraining**

```
if pretrain_history:
    plt.figure(figsize=(7, 4))
    plt.plot(range(1, len(pretrain_history) + 1), pretrain_history, marker='o')
    plt.title('SimSiam pretraining loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.grid(alpha=0.3)
    plt.show()
else:
    print('Loss plot skipped: set RUN_TRAINING=True to train in this notebook.')
```

شرط اول مانع رسم نمودار خالی می‌شود. محور افقی epoch و محور عمودی loss است. اگر آموزش داخل notebook انجام نشده باشد، history خالی است و پیام راهنما چاپ می‌شود.

سلول‌های ۱۵ و ۱۶: تست خطای checkpoint

> **هدف: اطمینان از مدیریت خطای قابل فهم**

```
from utils.checkpoint import load_checkpoint
try:
    load_checkpoint('checkpoints/file_that_does_not_exist.pt', SimSiam())
except FileNotFoundError as error:
    print('Expected error:', error)
    print('Missing-checkpoint handling: PASS')
```

`try` اجرای عملی را امتحان می‌کند. چون مسیر ساختگی است، تابع باید `FileNotFoundError` بدهد. `except` خطای مورد انتظار را می‌گیرد و نشان می‌دهد برنامه به‌جای crash نامفهوم، پیام راهنما دارد.

سلول‌های ۱۷ و ۱۸: Gradio Demo

> **هدف: اجرای رابط تعاملی مدل**

```
# from app.demo import launch
# launch('checkpoints/linear.pt')
```

این دو خط عمدی comment شده‌اند تا اجرای notebook به‌طور خودکار یک server دائمی باز نکند. برای اجرای دمو، علامت `#` را بردارید. تابع `launch` مدل را بارگذاری می‌کند، ورودی تصویر را می‌سازد و در آدرس `http://127.0.0.1:7860` اجرا می‌شود.

در `app/demo.py` تابع `predict` تصویر را به tensor تبدیل، به ۳۲×۳۲ resize، از encoder و classifier عبور و با softmax احتمال ۱۰ کلاس را تولید می‌کند.

نتیجه‌گیری و روش ارائه

1. ابتدا سلول setup و تست معماری را اجرا کنید.
2. شکل ورودی و خروجی اسکالر loss را توضیح دهید.
3. داده و augmentation را معرفی کنید.
4. تفاوت pretraining بدون label و linear evaluation با label را بگویید.
5. سلول ارزیابی را اجرا و دو accuracy را نمایش دهید.
6. در پایان demo را باز کنید و محدودیت کلاس‌های CIFAR-10 را توضیح دهید.

> **نتیجه مهم:** notebook برای ارائه آماده است؛ آموزش طولانی با `RUN_TRAINING=True` اختیاری است و checkpointهای فعلی امکان ارزیابی سریع را فراهم می‌کنند.